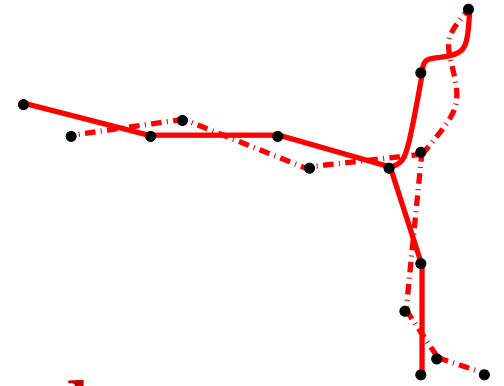
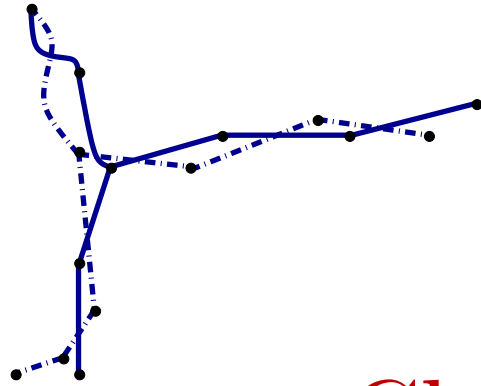
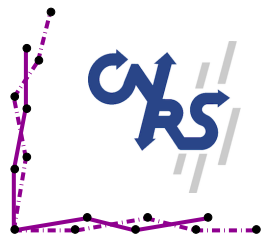




Cloud et Réseaux Virtuels

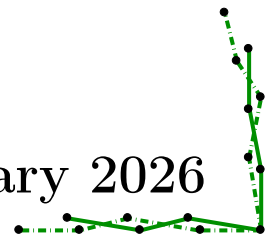
<https://perso.lip6.fr/binh-minh.bui-xuan/crv2025.html>

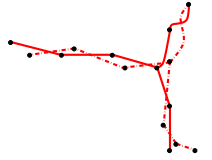
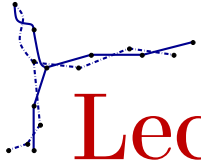
Binh-Minh Bui-Xuan



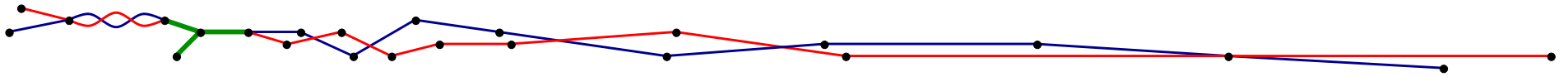
**SORBONNE
UNIVERSITÉ**
CRÉATEURS DE FUTURS
DEPUIS 1257

PARIS, February 2026





Lectures 6-10

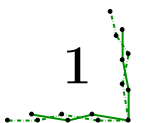
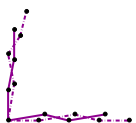


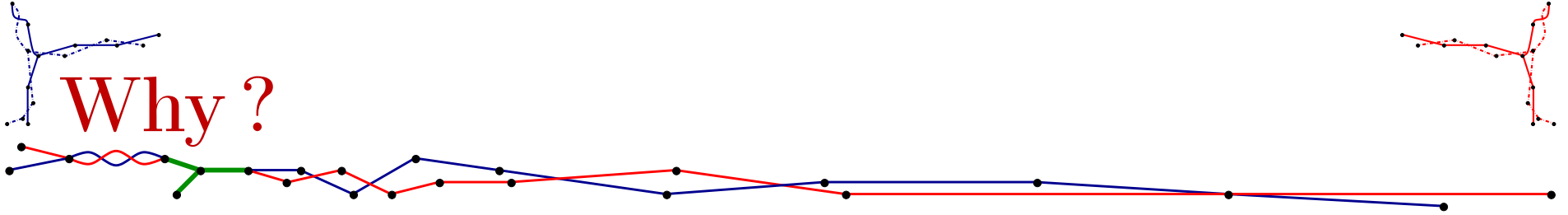
PROJECTS :

- project algo/prog
- project containerisation

TODAY WORK :

- IoT architecture
- Applications to robotics
- Cloud robotics : client-server model
- Reactive EDA (event-driven architecture) vs. procedural loops
- Monolithic vs. microservice architectures





Professional choices :

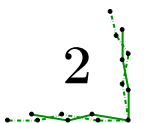
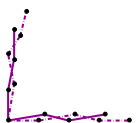
- Ingénieur DevOps, Intégrateur
- Data Engineer
- Ingénieur SRE, Ingénieur Système & Réseaux, DSI

Study choices :

- M2 : MU5IN552 DAAR (Moteurs de Recherche)

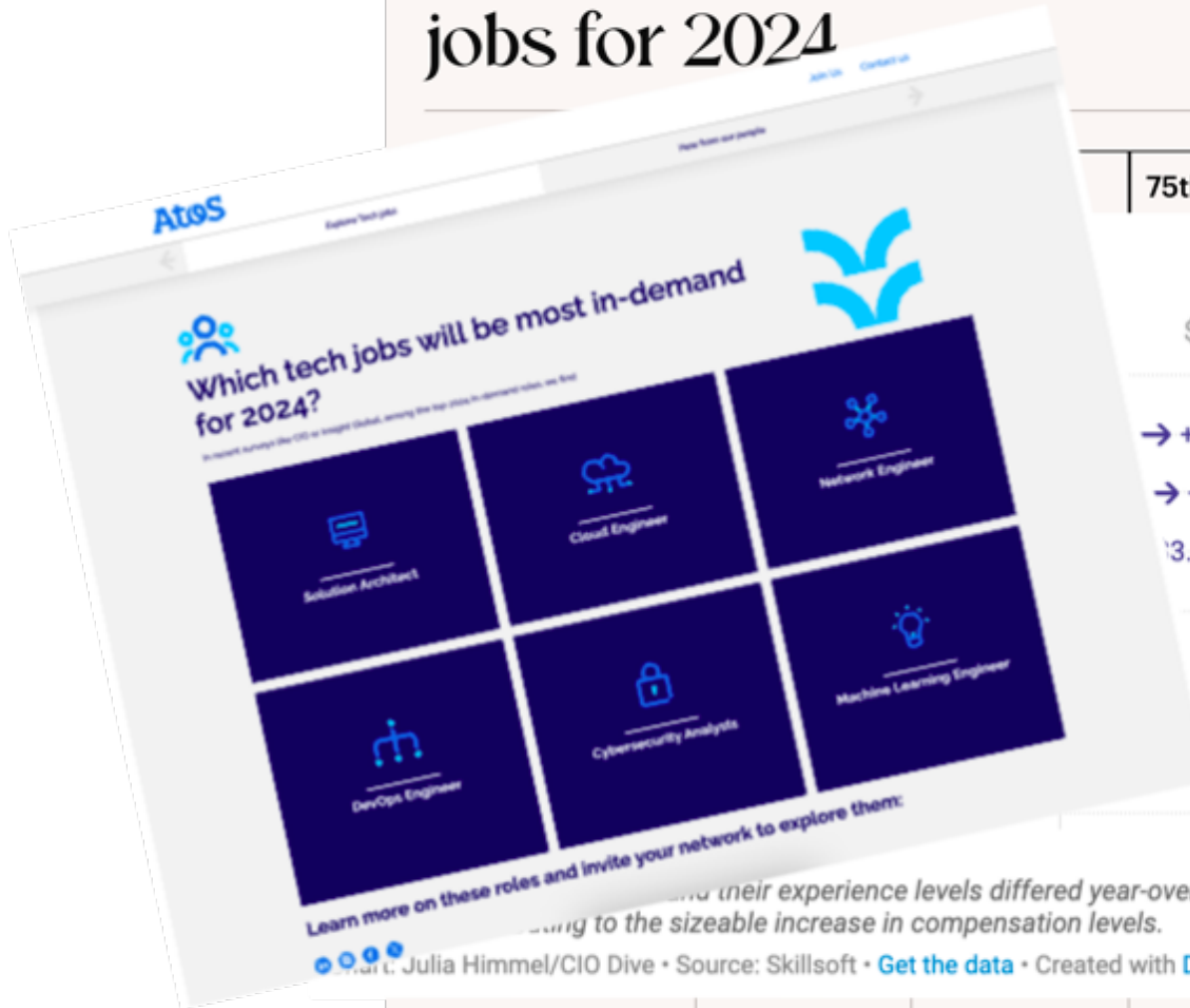
Scientific choices :

- EDITE
- ANR/ERC PI & partners'
- CIFRE



Why ?

The 15 most in-demand tech jobs for 2024



75th

\$120K \$140K \$160K

→ +\$8.2K

→ +\$36.6K

3.1K ←

-\$533 ←

→ +\$18.5K

→ +\$4.4K

→ +\$17.8K

→ +\$4.8K

Data security analyst

\$120,000

\$147,500

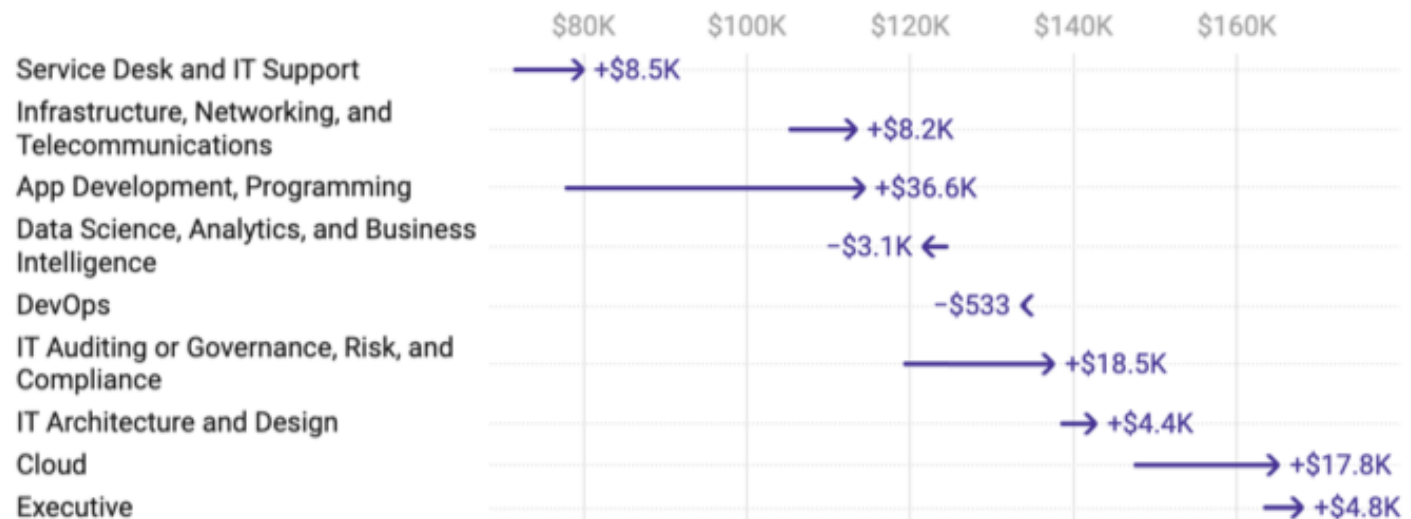
\$170,250

Why ?

The 15 most in-demand tech jobs for 2024

Job	25th	50th	75th
-----	------	------	------

Annual compensation by role in 2022, 2023



The number of respondents and their experience levels differed year-over-year for the app development, programming job roles, contributing to the sizeable increase in compensation levels.

Chart: Julia Himmel/CIO Dive • Source: Skillsoft • [Get the data](#) • Created with [Datawrapper](#)

Job	25th	50th	75th
Data security analyst	\$120,000	\$147,500	\$170,250

Why ?

The 15 most in-demand tech jobs for 2024

Job	25th percentile	50th percentile	75th percentile
Systems security manager	\$137,250	\$167,750	\$198,500
Network/cloud architect	\$133,000	\$165,750	\$196,750
Applications architect	\$134,750	\$163,250	\$195,500
IT director	\$132,000	\$161,250	\$192,000
ERP integration manager	\$120,500	\$153,250	\$187,500
Big data engineer	\$122,000	\$149,500	\$174,250
Data security analyst	\$120,000	\$147,500	\$170,250

40K \$160K

+\$18.5K

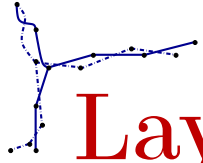
→ +\$4.4K

→ +\$17.8K

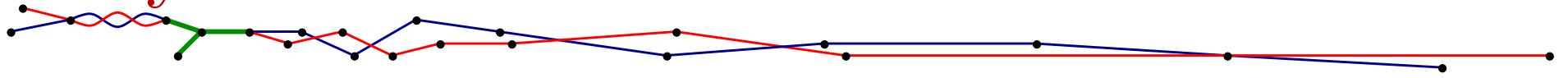
→ +\$4.8K

to development, programming

IoT architecture



Layers in an IoT architecture

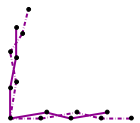


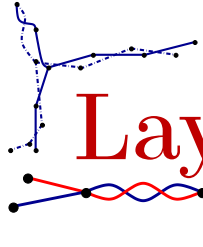
4. Business

3. Processing

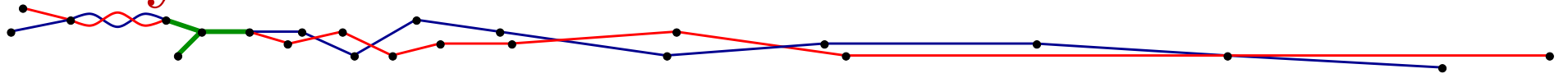
2. Transport

1. Perception

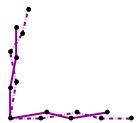


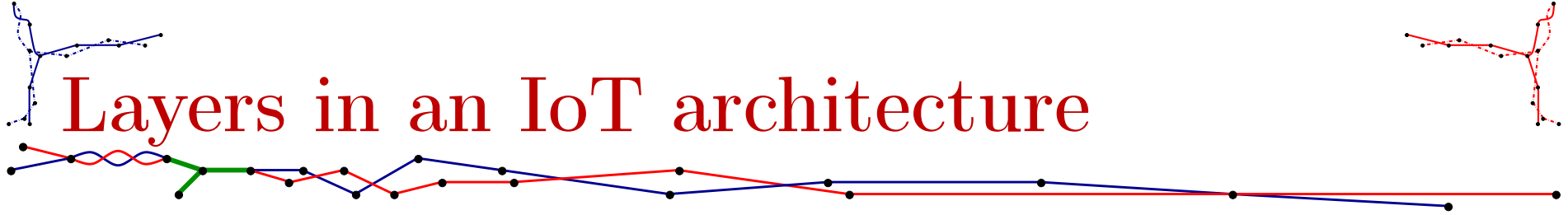


Layers in an IoT architecture



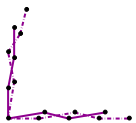
4. Business $\Rightarrow$ software development
3. Processing $\Rightarrow$ data engineering
2. Transport $\Rightarrow$ network technologies
1. Perception $\Rightarrow$ electronics





Layers in an IoT architecture

4. Business $\Rightarrow$ software development + hybrid needs
3. Processing $\Rightarrow$ data engineering + scaling up
2. Transport $\Rightarrow$ network technologies + encryption
1. Perception $\Rightarrow$ electronics + regulations



Layers in an IoT architecture

IoT 4 layers model

4 Integrated Applications



3 Information Processing



2 Network Infrastructure



1 Sensing and Identification



[1] Tabbane, 2019

Layers in an IoT architecture

IoT 4 layers model

4 Integrated Applications



3 Information Processing



2 Network Infrastructure

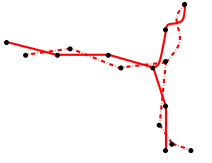


1 Sensing and Identification

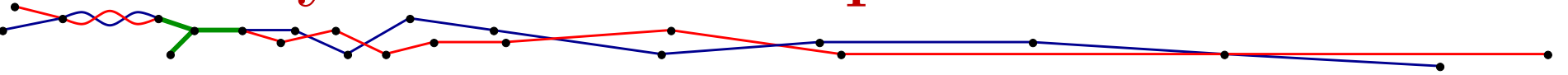


S
E
C
U
R
I
T
Y

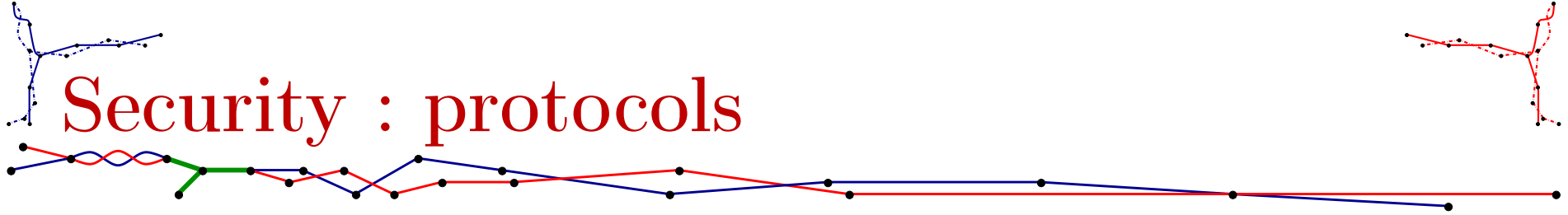
[1] Tabbane, 2019



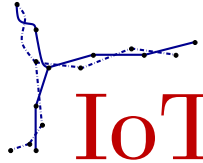
Security : threats and protections



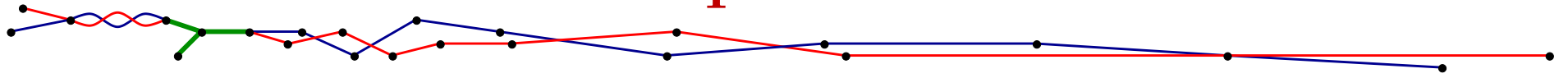
THREAT	PROTECTION
Phishing	Network stability
Malware	Sniffing
Denial-of-Service	Retry policy
DNS Tunneling	Scalability
Spoofing	Man in the middle
Code Injection	...
Supply Chain Attacks	
...	



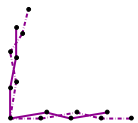
PROTECTION	HTTP	HTTPS	KAFKA	MQTT
Network stability	😊	😊	😞	😊
Sniffing	😞	😊	😊	😊
Retry policy	😞	😞	😊	😊
Scalability	😞	😞	😊	😞
Man in the middle	😞	😞	😞/😊	😊
...				



IoT architecture : protocols



4. Business $\Rightarrow$ software development + hybrid needs
3. Processing $\Rightarrow$ data engineering + scaling up
2. Transport $\Rightarrow$ network technologies + encryption
1. Perception $\Rightarrow$ electronics + regulations



IoT architecture : protocols

IoT 4 layers model

4 Integrated Applications



3 Information Processing



2 Network Infrastructure



1 Sensing and Identification



HTTPS
MQTT

MQTT

RF

S
E
C
U
R
I
T
Y

[1] Tabbane, 2019

IoT architecture : protocols

IoT 4 layers model

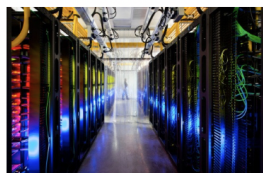
4 Integrated Applications



3 Information Processing



2 Network Infrastructure



1 Sensing and Identification



HTTPS
MQTT

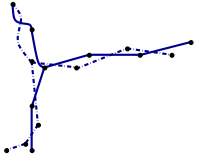
KAFKA

MQTT

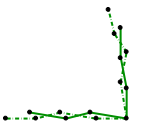
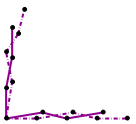
RF

S
E
C
U
R
I
T
Y

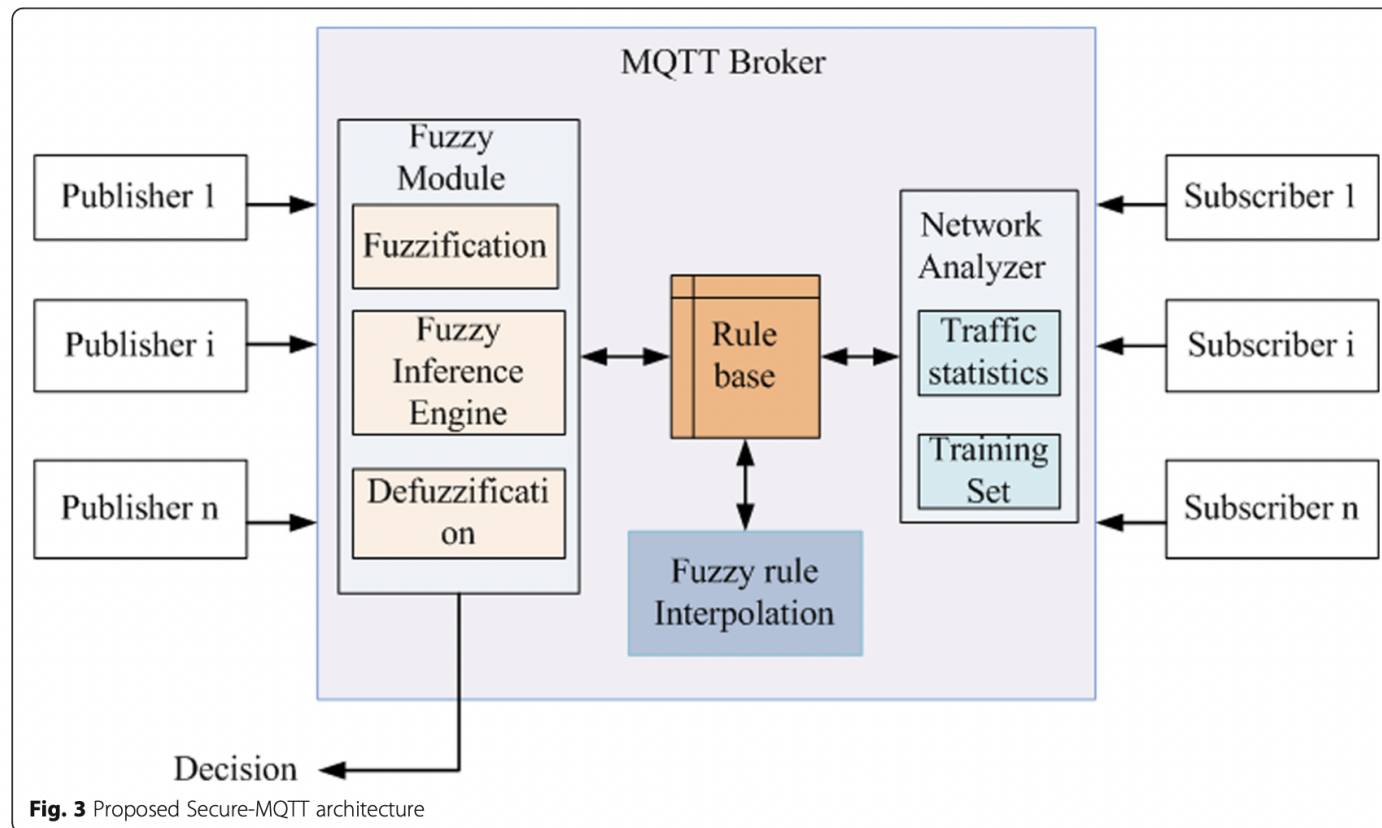
[1] Tabbane, 2019



Research Topics

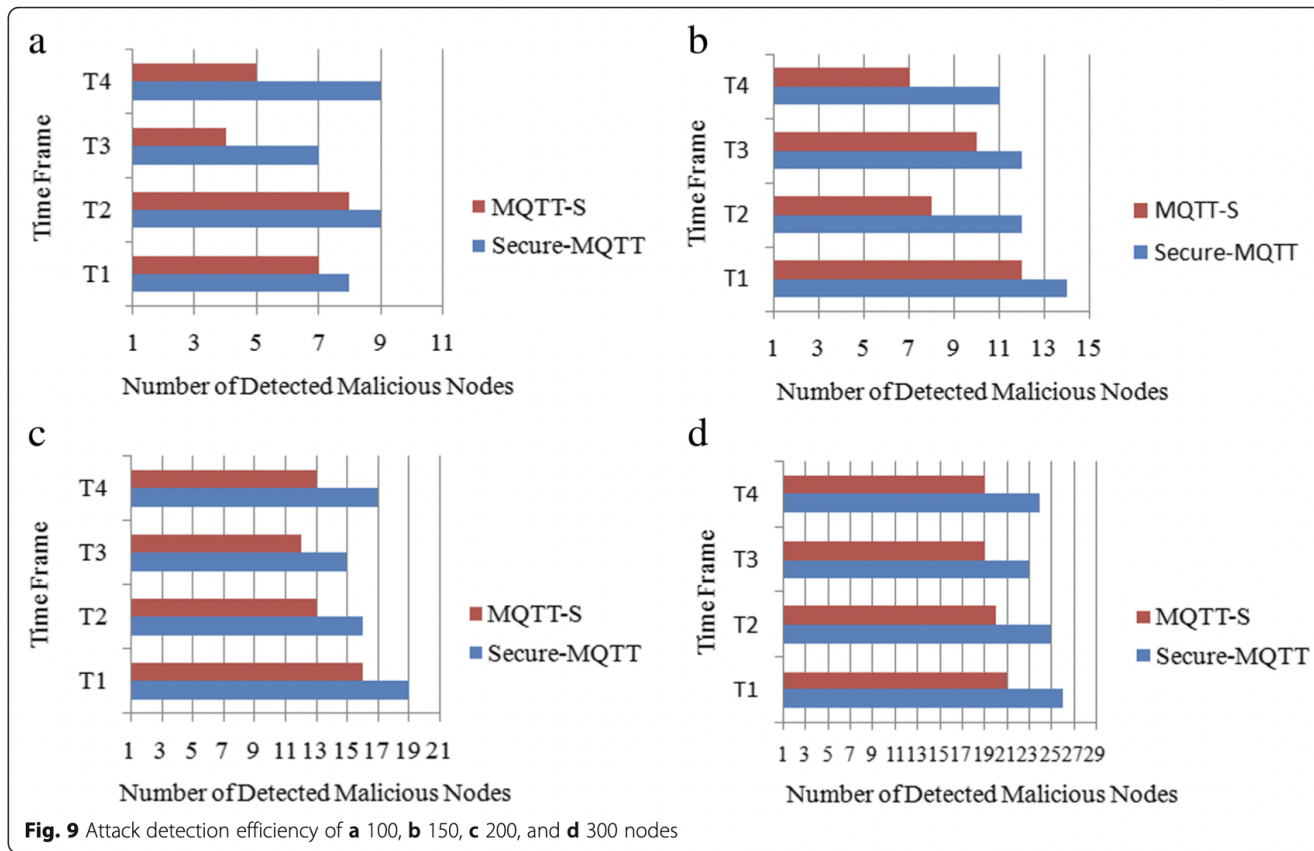


MQTT $\Rightarrow$ MQTT-S $\Rightarrow$ increased security



[2] Haripriya, Kulothungan, 2019 : fuzzy logic vs. DoS

MQTT $\Rightarrow$ MQTT-S $\Rightarrow$ increased security



[2] Haripriya, Kulothungan, 2019 : fuzzy logic vs. DoS

Data lineage MQTT/RFID

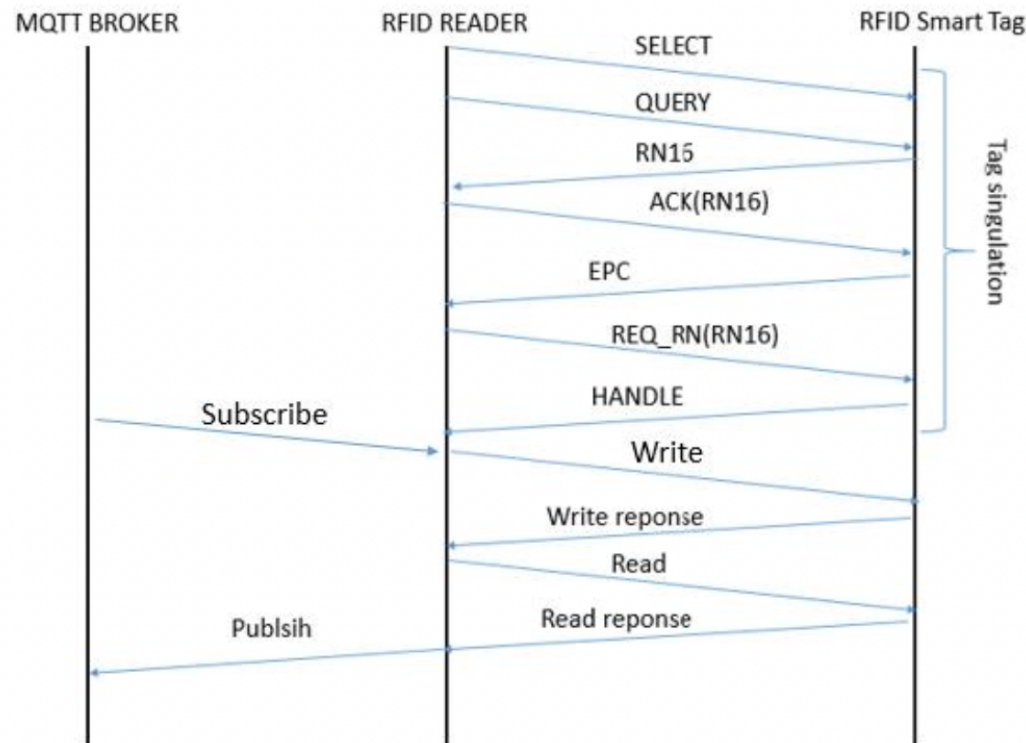


Figure 4. The proposed system based on publish/suscribe pattern

[3] Arif et al., 2019 : use RFID in logistics outsourcing

KAFKA $\Rightarrow$ scalable data lineage

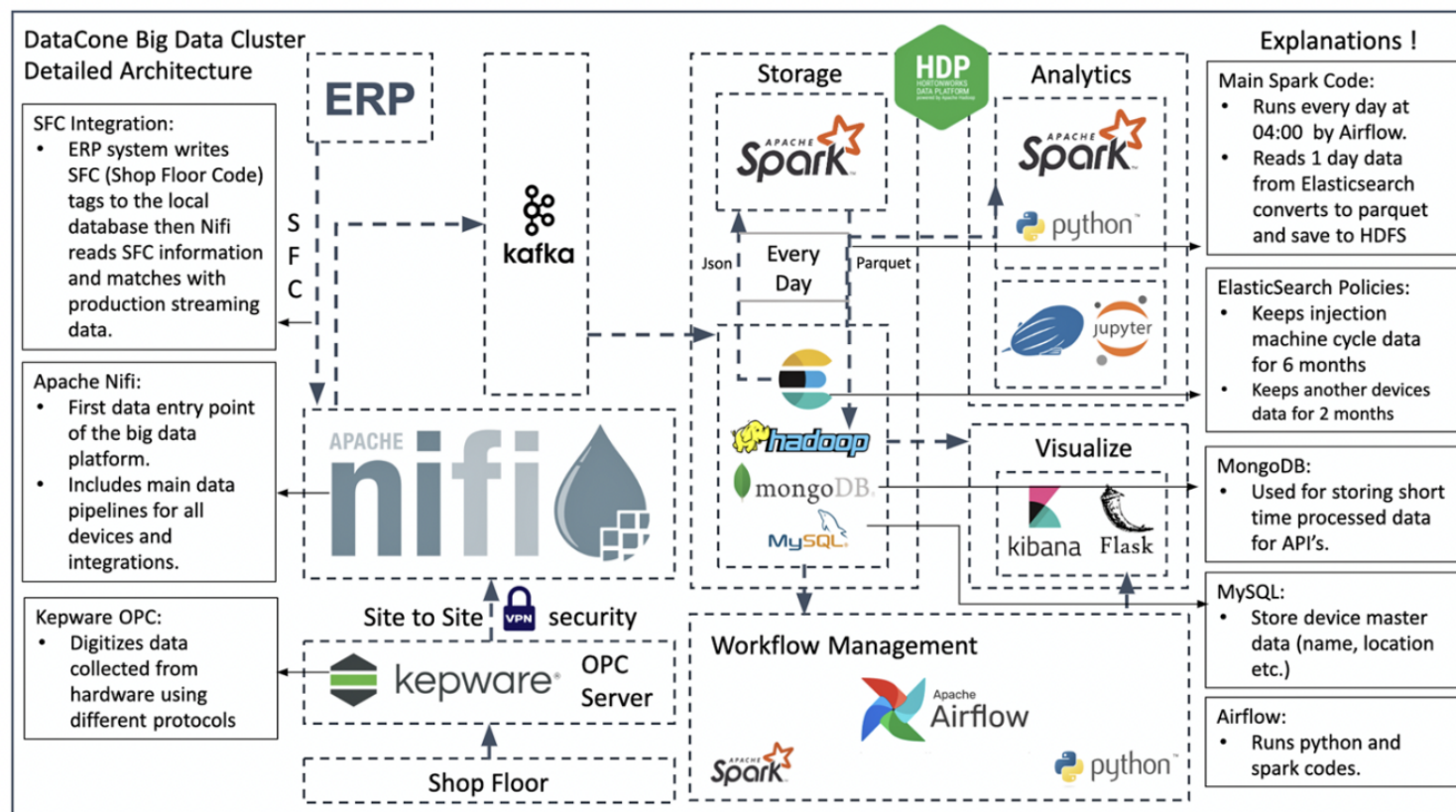


Fig. 2 Detailed architecture design of DataCone with a combination of proper open-source big data

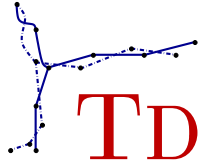
[4] Cakir et al., 2022

KAFKA $\Rightarrow$ scalable data lineage

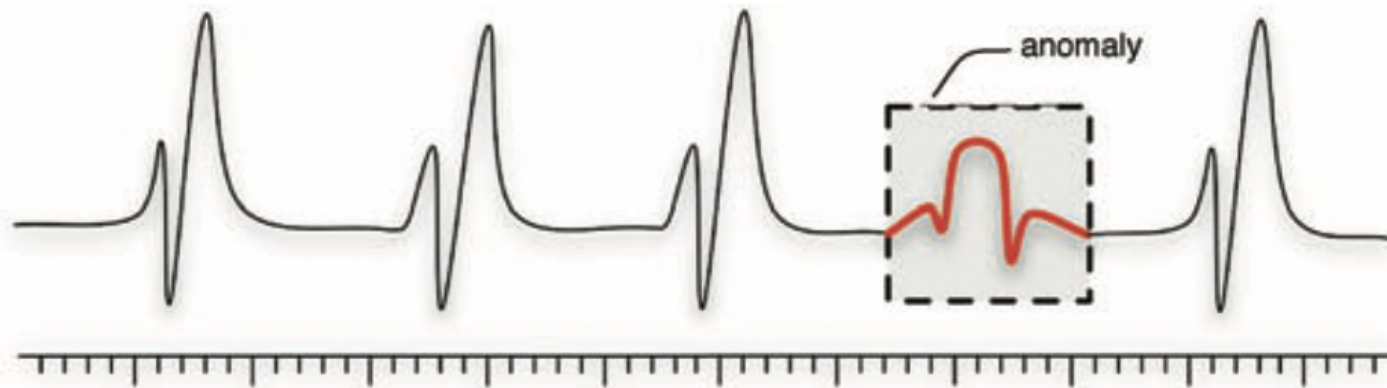
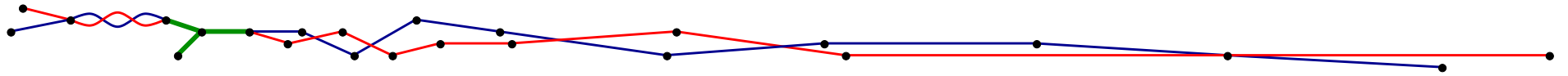
	INTERFACES	ODBC	MQTT, REST
	PLUG-IN	Kepware - KepserverEX	Kepware - KepserverEX
Ingest	SERVER	Kepware - KepserverEX	
Connect	DRIVERS	PLC, Euromap63, OPC Client, RTU, Database, Custom	
	DEVICES	Analyzer, Sensor, Injection Molding Machine, PLC, Robot,	

Data Flow Direction

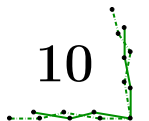
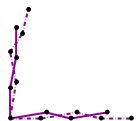
[4] Cakir et al., 2022

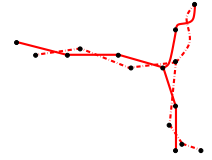
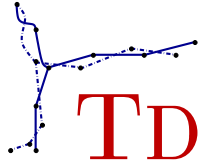


TDB : time series databases

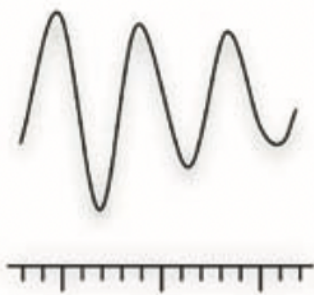
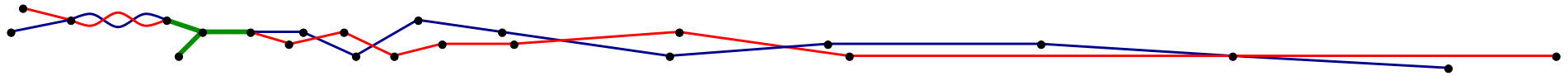


[5] Esling, Agon, 2012

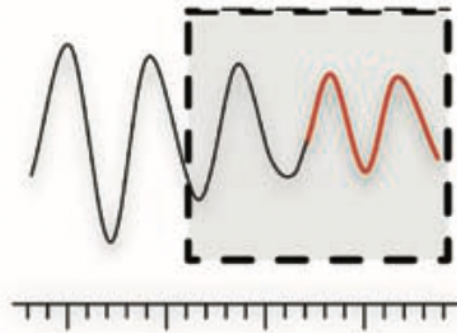




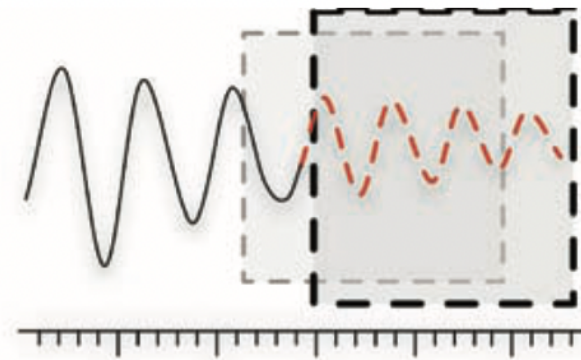
TDB : time series databases



(a)

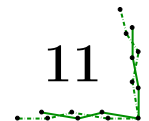
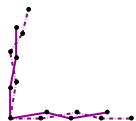


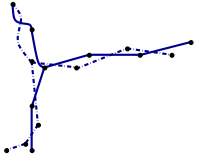
(b)



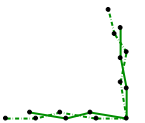
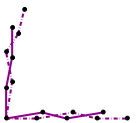
(c)

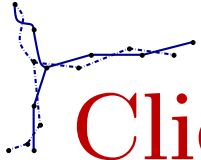
[5] Esling, Agon, 2012



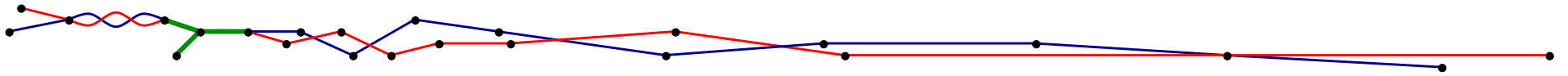


Cloud robotics, edge/cloud continuum



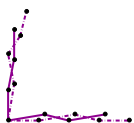


Client-server architecture

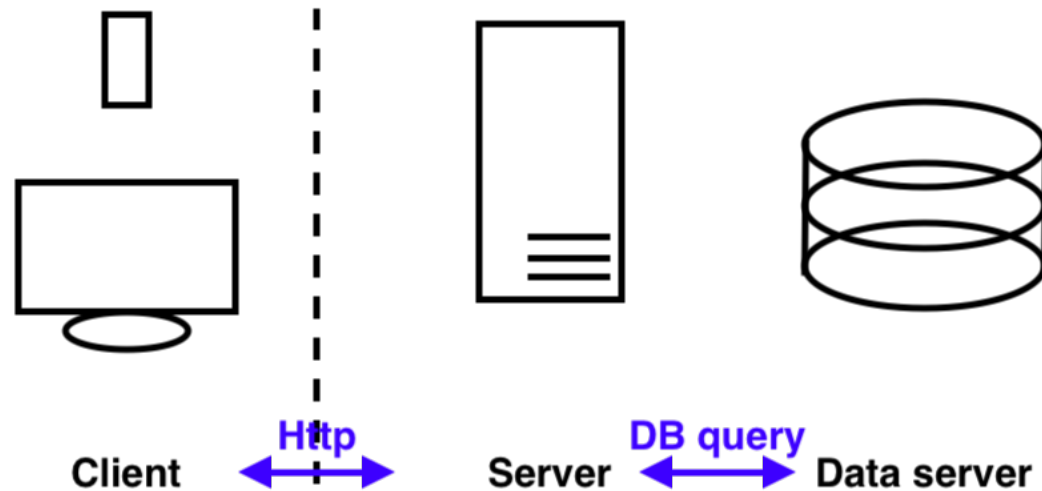


HIGHLIGHTS :

- n -tiers, three layers, client-server model
- endpoints, paths, API
- other usecase : cloud robotics
- reactive EDA (event-driven architecture) vs. procedural loops
- monolith vs. microservices

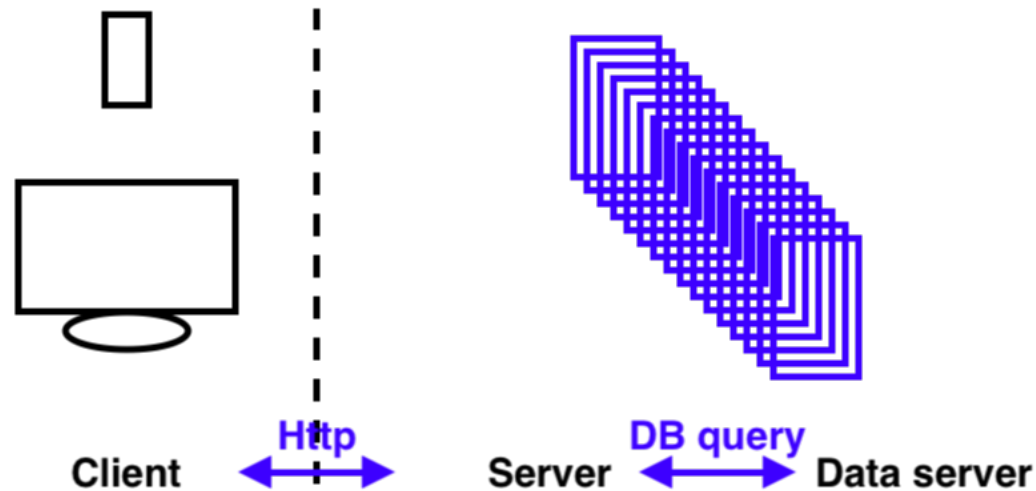


Client-server model : three layers

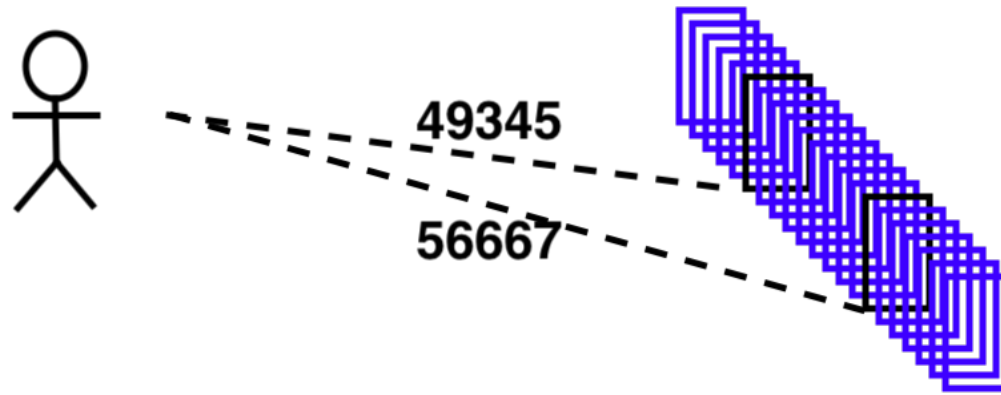
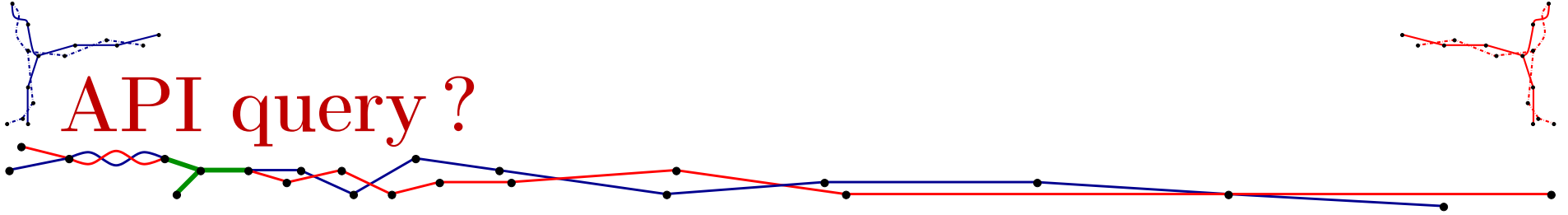


- Numerous client machines → Http traffic
- Some servers (controllers if MVC ; brookers ; load balancers ; gateways ; ...)
- Fewer data servers (on premises) or numerous data servers (on cloud)

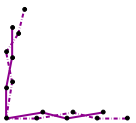
Client-server model : three layers

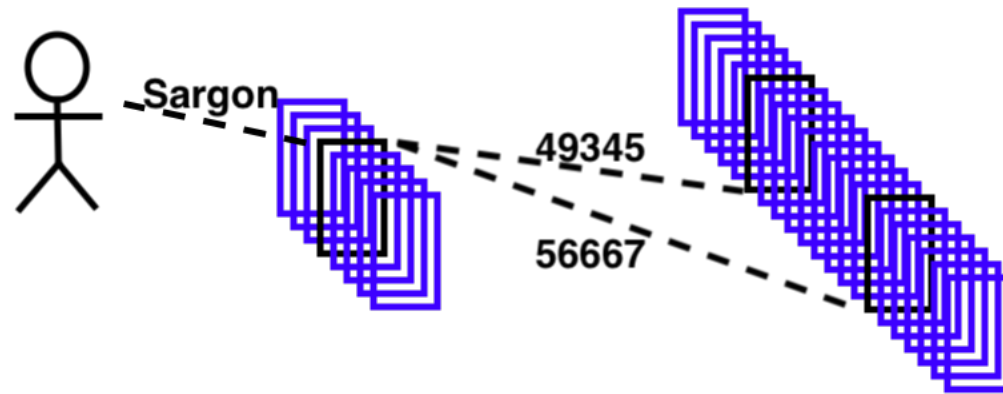


- API view : response in standardized format
- Example : `http://www.gutenberg.org/ebooks/` → list of all books
- `http://www.gutenberg.org/ebooks/id` → details of book with pk equals id



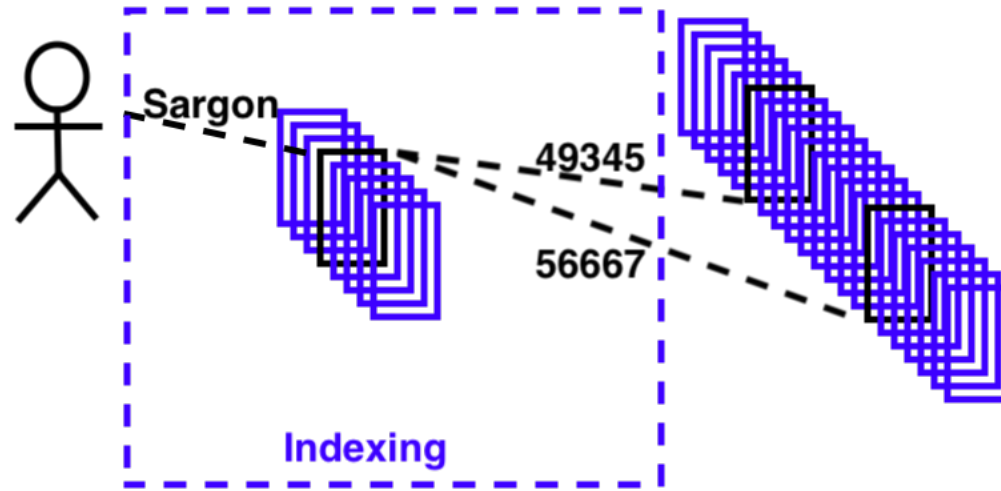
- User asks 49345 ; Gutenberg responses *History of Babylonia vol. 1*
- User asks 56667 ; Gutenberg responses *History of Babylonia vol. 2*
- Issue : meaning of 49345 and 56667 to a human ? !





- Want : user asks Sargon for a list of relevant books, e.g. 49345 and 56667
- Choice : middle man server vs. client-side computation ?
- Issue : computing time ? precomputing ? real-time response ?

API query ? Redirection ? Indexing ?



- Indexing : precompute part of the answers, e.g. keywords, names, ...
- Wiring : each precomputed answer links to the original document
- Issue : space explosion ? automatic update ?

Other usecase : cloud robotics



Other usecase : cloud robotics



Hardware power!

Other usecase : cloud robotics



Hardware power!

Hardware power!

Hardware power!

Hardware power!

Other usecase : cloud robotics



Soft?

Hardware power!

Hardware power!

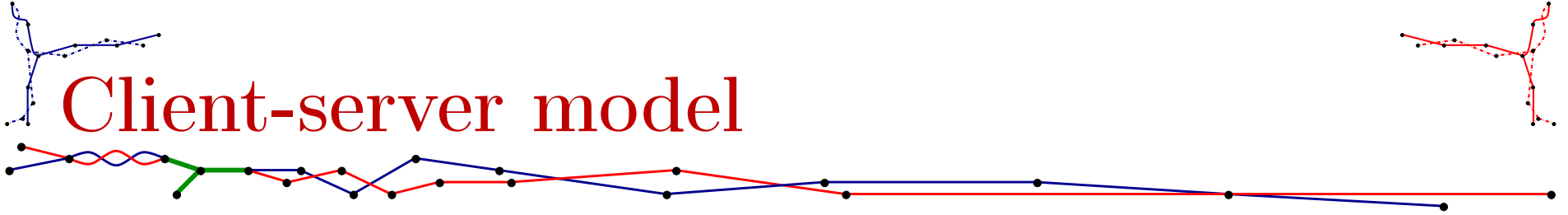
Hardware power!

Hardware power!

- Software? ← not only simple scripts
- Distributed systems
- Must operate in real world conditions

Other usecase : cloud robotics





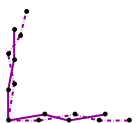
Client : robots

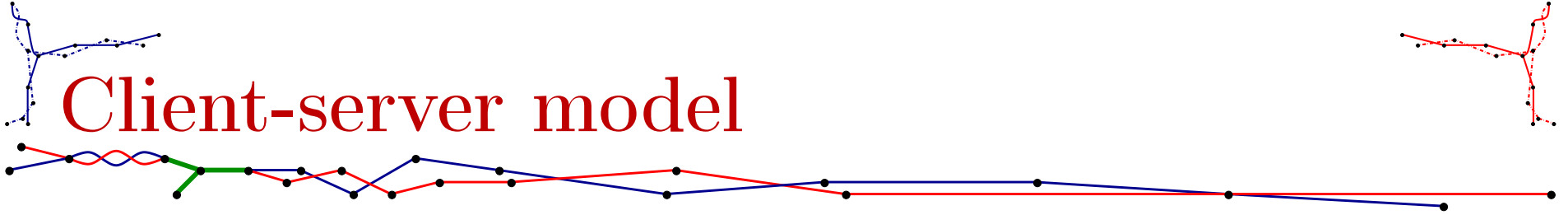
- send data to server (sensors)
- avoid processing data
- receive instructions
- perform instructions (actuators)

⇒ look-compute-move (LCM) while avoiding the computing part

Robots $\approx$ small brain vs. servers $\approx$ supercomputers (heat, energy)

SIMILARITY : LLMs.





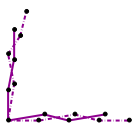
Client : robots

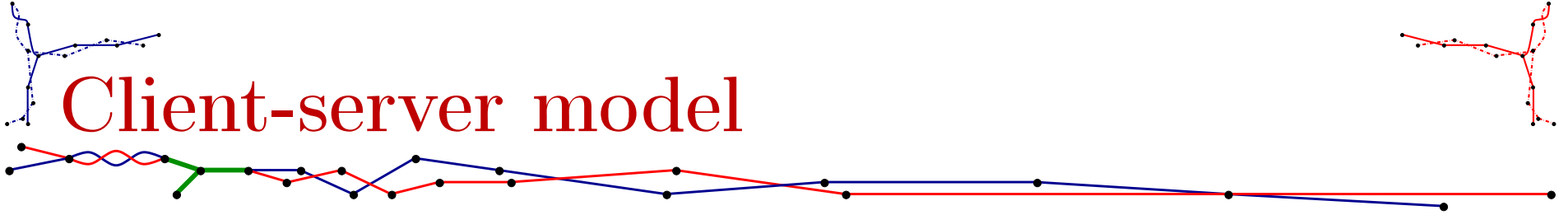
- send data to server (sensors)
- avoid processing data
- receive instructions
- perform instructions (actuators)

⇒ look-compute-move (LCM) while avoiding the computing part

Robots $\approx$ small brain vs. servers $\approx$ supercomputers (heat, energy)

QUESTION : always a good idea ?





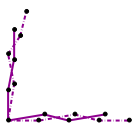
Client : robots

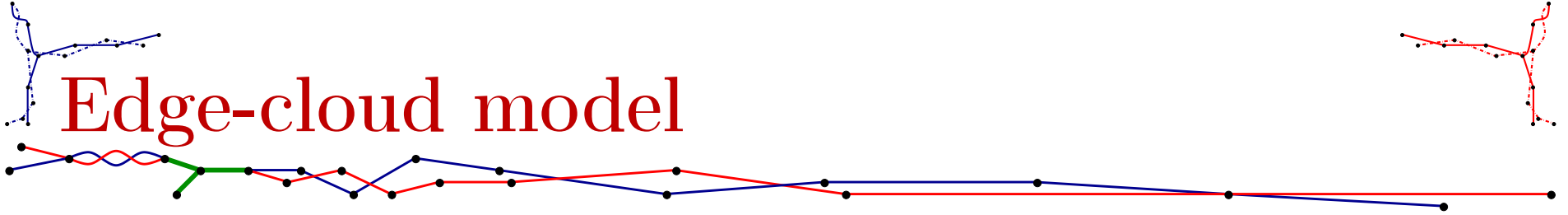
- send data to server (sensors)
- avoid processing data
- receive instructions
- perform instructions (actuators)

⇒ look-compute-move (LCM) while avoiding the computing part

Robots $\approx$ small brain vs. servers $\approx$ supercomputers (heat, energy)

NB : only for non-critical decisions !



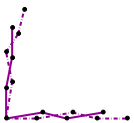


Edge : robots

- safety-critical actions
- low latency, reliability tasks
- similar to “reflexes”

Cloud : supercomputers, datacenters, ...

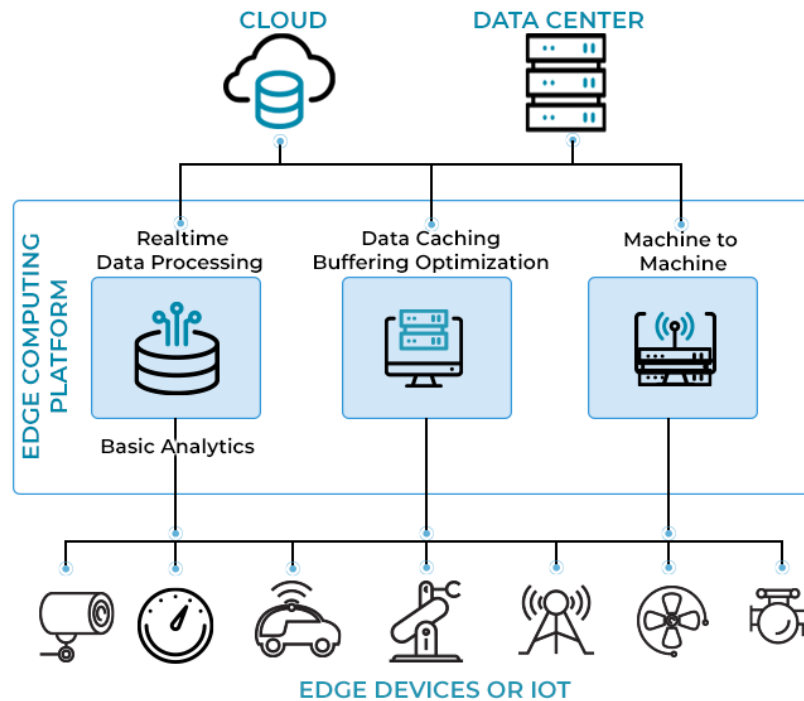
- efficiency, optimisation, machine learning tasks
- heavy search, search in distributed databases
- similar to “neurons”

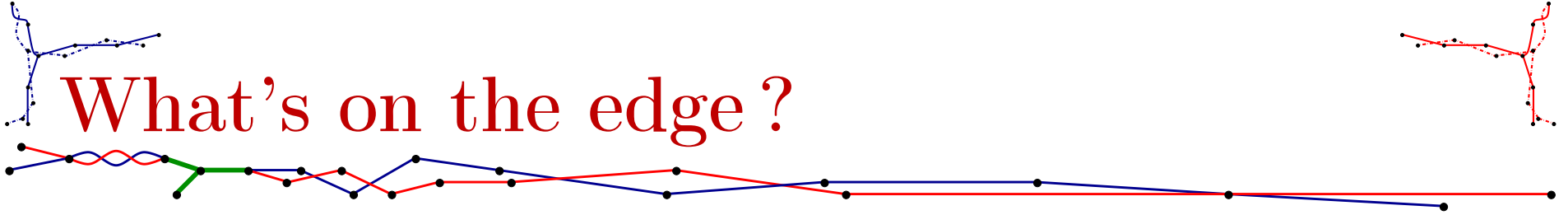


Edge computing



EDGE COMPUTING





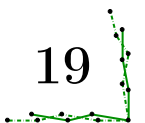
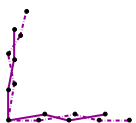
What's on the edge ?

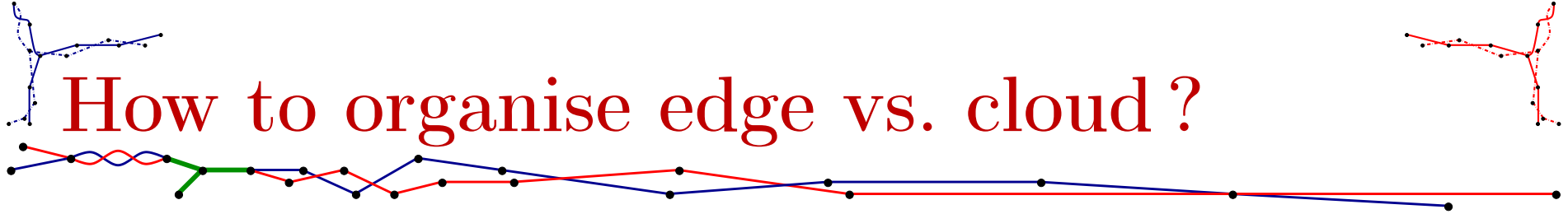
Critical examples :

- E-STOP (emergency stop)
- stabilizing UAV, motor cooling : need high frequency feedbacks
- raw sensor preprocessing : filtering, movement detection, ...

But also :

- odometry : estimation of the position without GPS
- image resolution adjustments : save bandwidth





How to organise edge vs. cloud ?

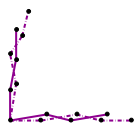
No edge $\approx$ sequential algorithms :

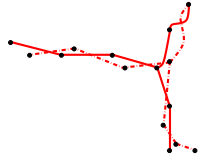
- I/O
- great for typing, specification, proofs
- cannot interrupt

With edge $\approx$ distributed algorithms :

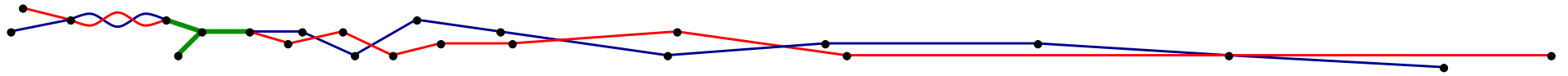
- sync vs. async
- concurrency $\Rightarrow$ significantly more complex proofs
- interrupt managements : can use events !

$\rightarrow$ reactive programming, EDA (event-driven architecture)



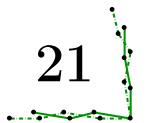
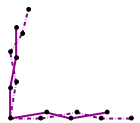


Reactive EDA



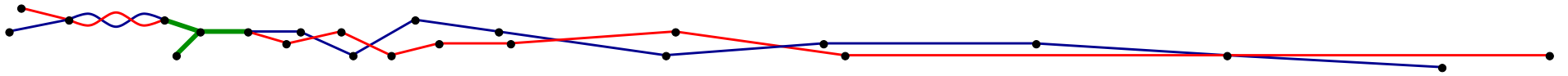
Main features :

- reactive vs. procedural
- event bus vs. polling
- publish/subscribe design pattern





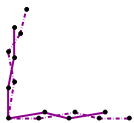
Reactive EDA



Main features :

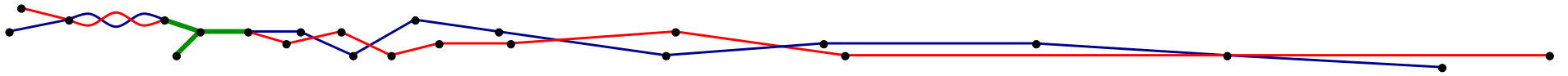
- reactive vs. procedural
- event bus vs. polling
- publish/subscribe design pattern

QUESTION : why reactive ?





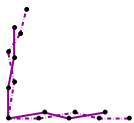
Reactive EDA



Main features :

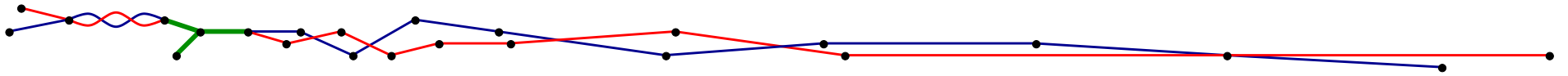
- reactive vs. procedural
- event bus vs. polling
- publish/subscribe design pattern

QUESTION : why reactive ? → critical, millisecond-level tasks





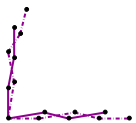
Reactive EDA



Main features :

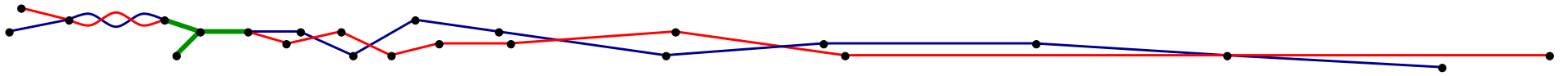
- reactive vs. procedural
- event bus vs. polling
- publish/subscribe design pattern

QUESTION : why not polling ?





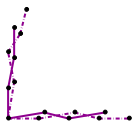
Reactive EDA



Main features :

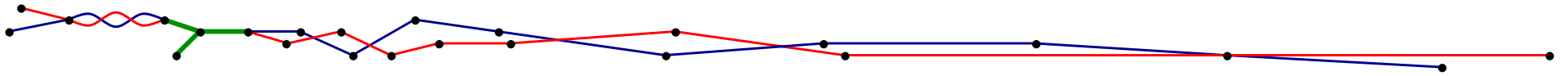
- reactive vs. procedural
- event bus vs. polling
- publish/subscribe design pattern

QUESTION : why not polling ? → save bandwidth !





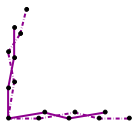
Reactive EDA



Main features :

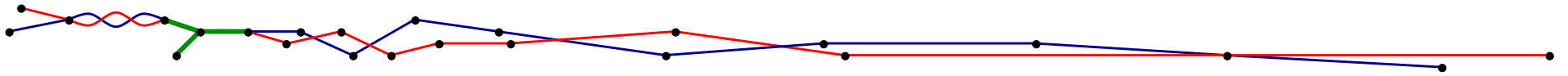
- reactive vs. procedural
- event bus vs. polling
- publish/subscribe design pattern

QUESTION : why do pub/sub ?





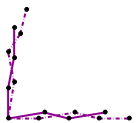
Reactive EDA



Main features :

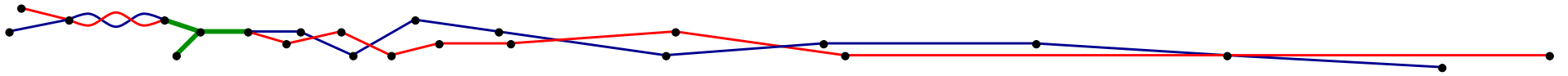
- reactive vs. procedural
- event bus vs. polling
- publish/subscribe design pattern

QUESTION : why do pub/sub ? → CPU can sleep (save battery)





Reactive EDA

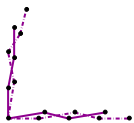


Main features :

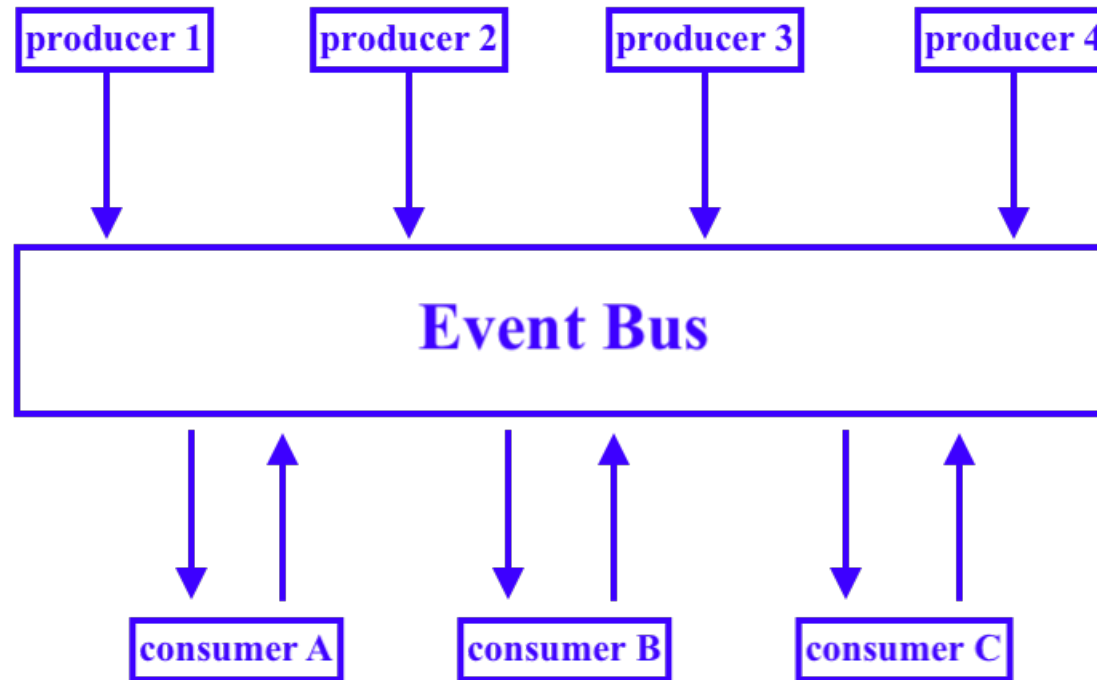
- reactive vs. procedural
- event bus vs. polling
- publish/subscribe design pattern

EXAMPLES OF EDA :

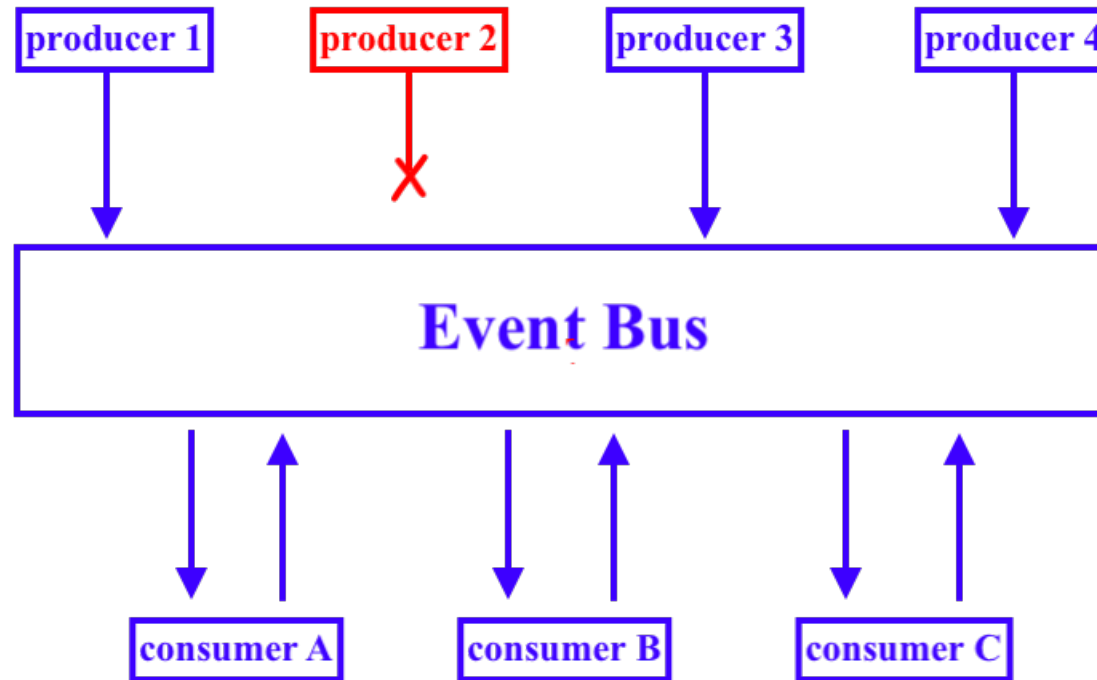
- ROS, Mosquitto, Kafka, Pulsar, Event Hubs, ...

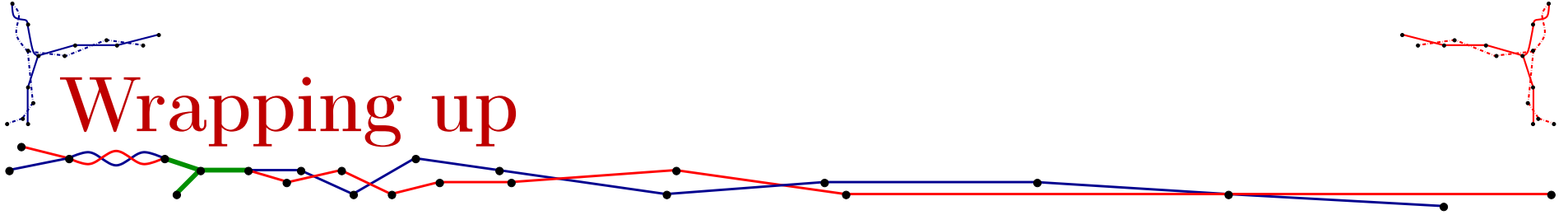


Fault tolerance with EDA



Fault tolerance with EDA





Client server (Edge/Cloud) :

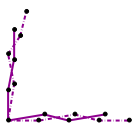
- split computation robot vs. remote server

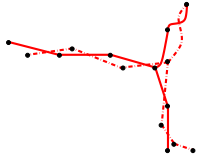
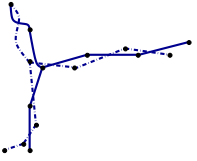
Event driven architecture (EDA Pub/Sub) :

- asynchronous communication
- real time systems

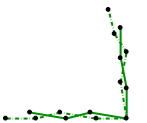
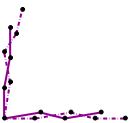
Microservices, nodes, modules :

- independent processes
- fault tolerance





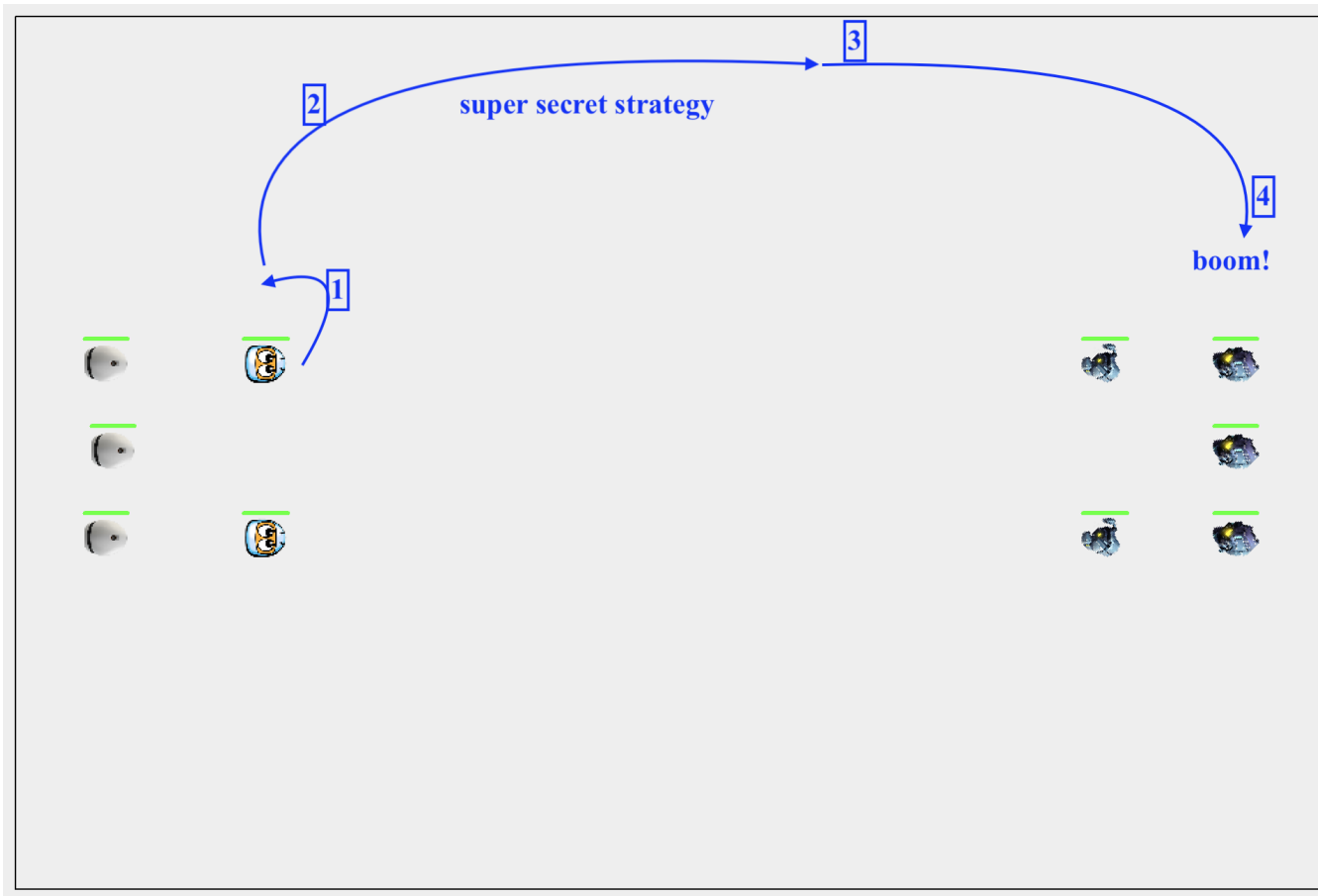
Preparation for lab sessions
(and project algo/prog)



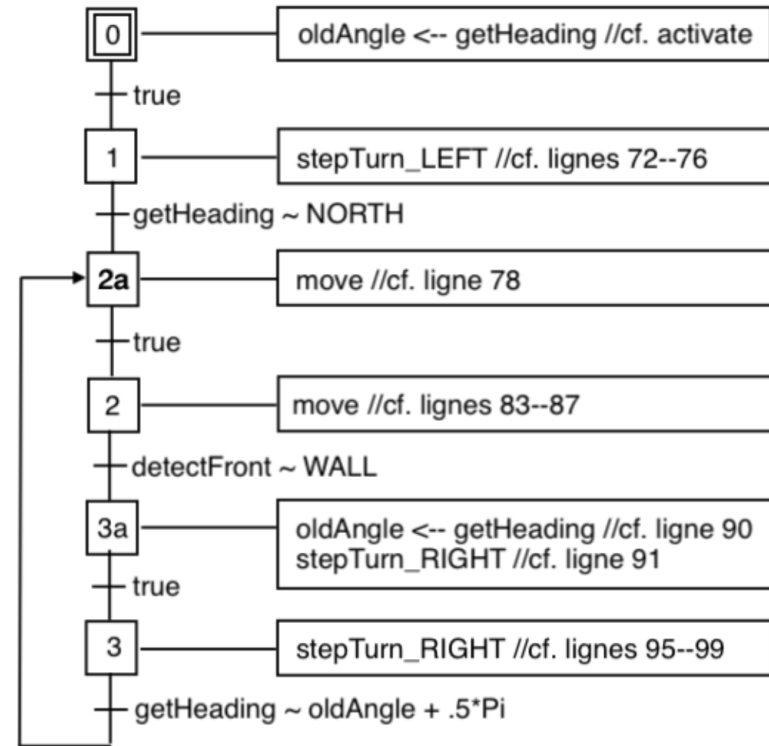
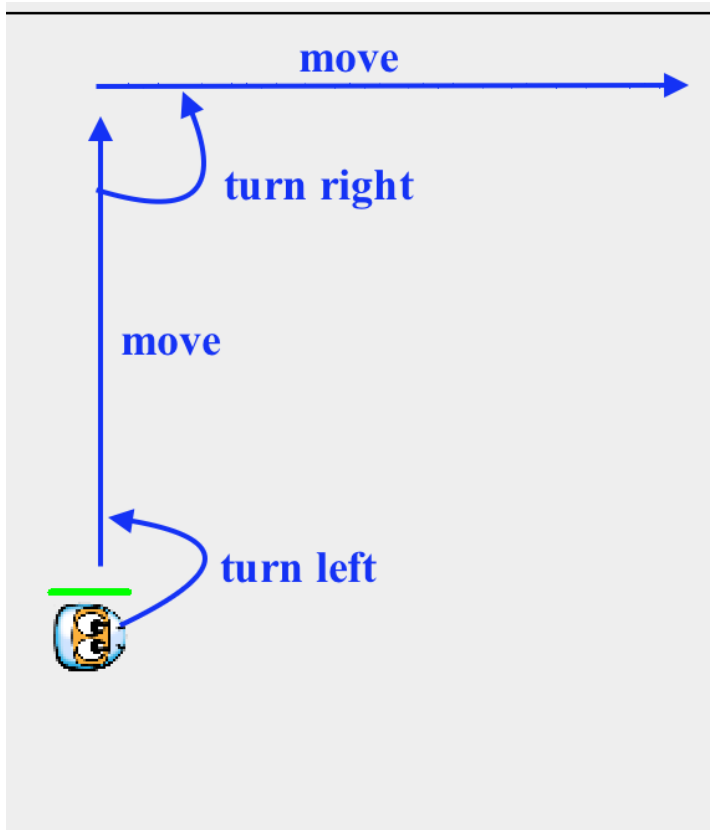
Example with simovies simulator



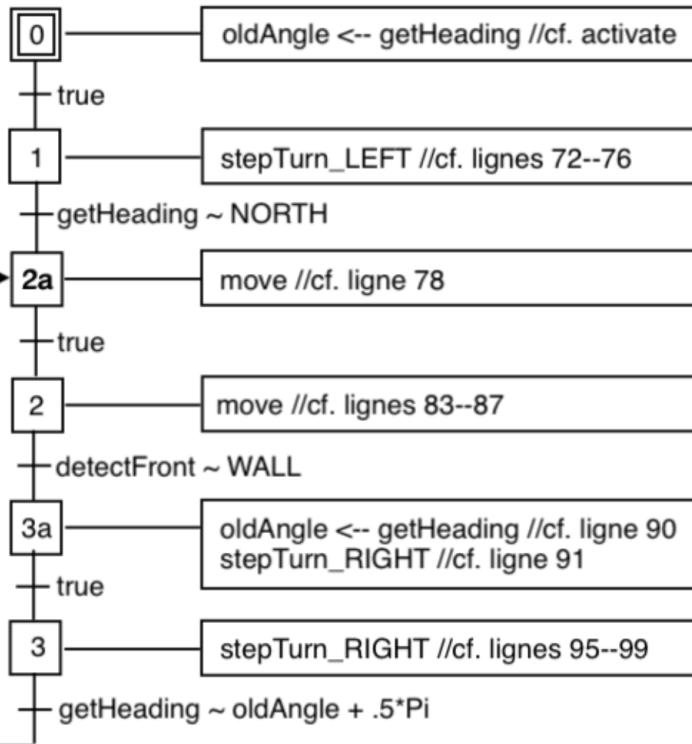
Example with simovies simulator



From conception to grafcet



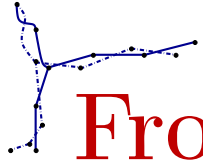
From grafcet to code



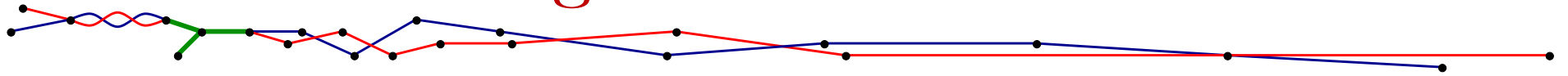
//AUTOMATON

```

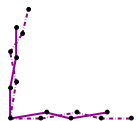
if (state==TURNLEFTTASK && !(isSameDirection(getHeading(),Parameters.NORTH))) {
    stepTurn(Parameters.Direction.LEFT);
    return;
}
if (state==TURNLEFTTASK && isSameDirection(getHeading(),Parameters.NORTH)) {
    state=MOVETASK;
    myMove();
    return;
}
if (state==MOVETASK && detectFront().getObjectType()!=IFrontSensorResult.Types.WALL) {
    myMove();
    return;
}
if (state==MOVETASK && detectFront().getObjectType()==IFrontSensorResult.Types.WALL) {
    state=TURNRIGHTTASK;
    oldAngle=getHeading();
    stepTurn(Parameters.Direction.RIGHT);
    return;
}
if (state==TURNRIGHTTASK && !(isSameDirection(getHeading(),oldAngle+Parameters.RIGHTTURN))) {
    stepTurn(Parameters.Direction.RIGHT);
    return;
}
if (state==TURNRIGHTTASK && isSameDirection(getHeading(),oldAngle+Parameters.RIGHTTURN)) {
    state=MOVETASK;
    myMove();
    return;
}
  
```

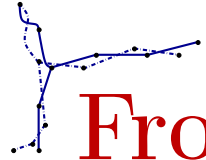



From code to grafcet

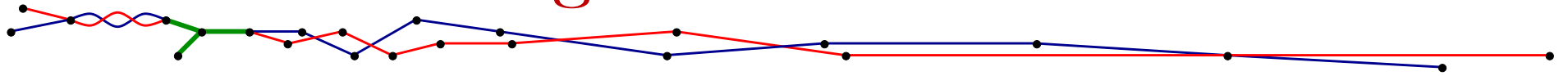


???



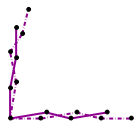


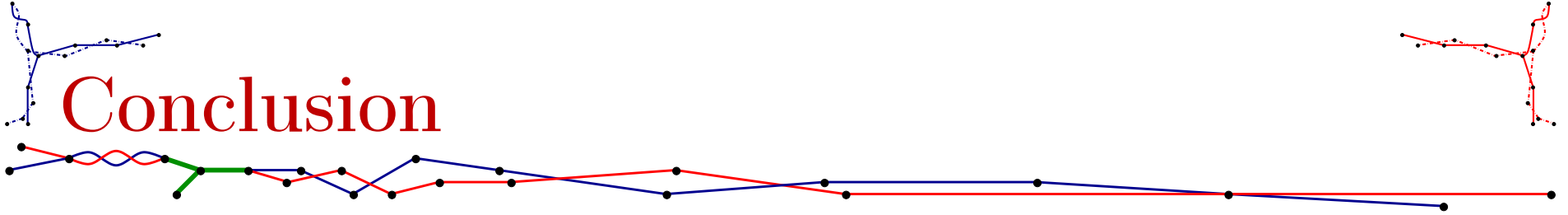
From code to grafcet



Reverse engineering, decompilation :

- more complex
- usually impossible



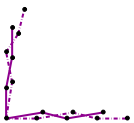


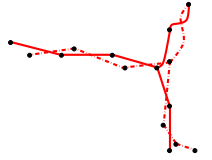
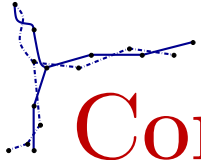
Conclusion

IoT architecture :

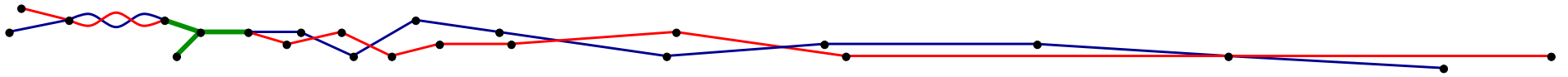
- ▷ 4 layers + security (5th level, cross-layer layer)
- ▷ RF, MQTT, HTTP(S) : network slices
- ▷ KAFKA, TDB : algorithms

⇒ cross-engineering skills : electronics, netx, data, software, ...





Conclusion



CLOUD ROBOTICS :

- On cloud robotics : learning, search, big data optimisation, ...
- On edge robotics : safety critical reactivity.
- EDA + client-server.

⇒ enjoy TME6 practical works !

